

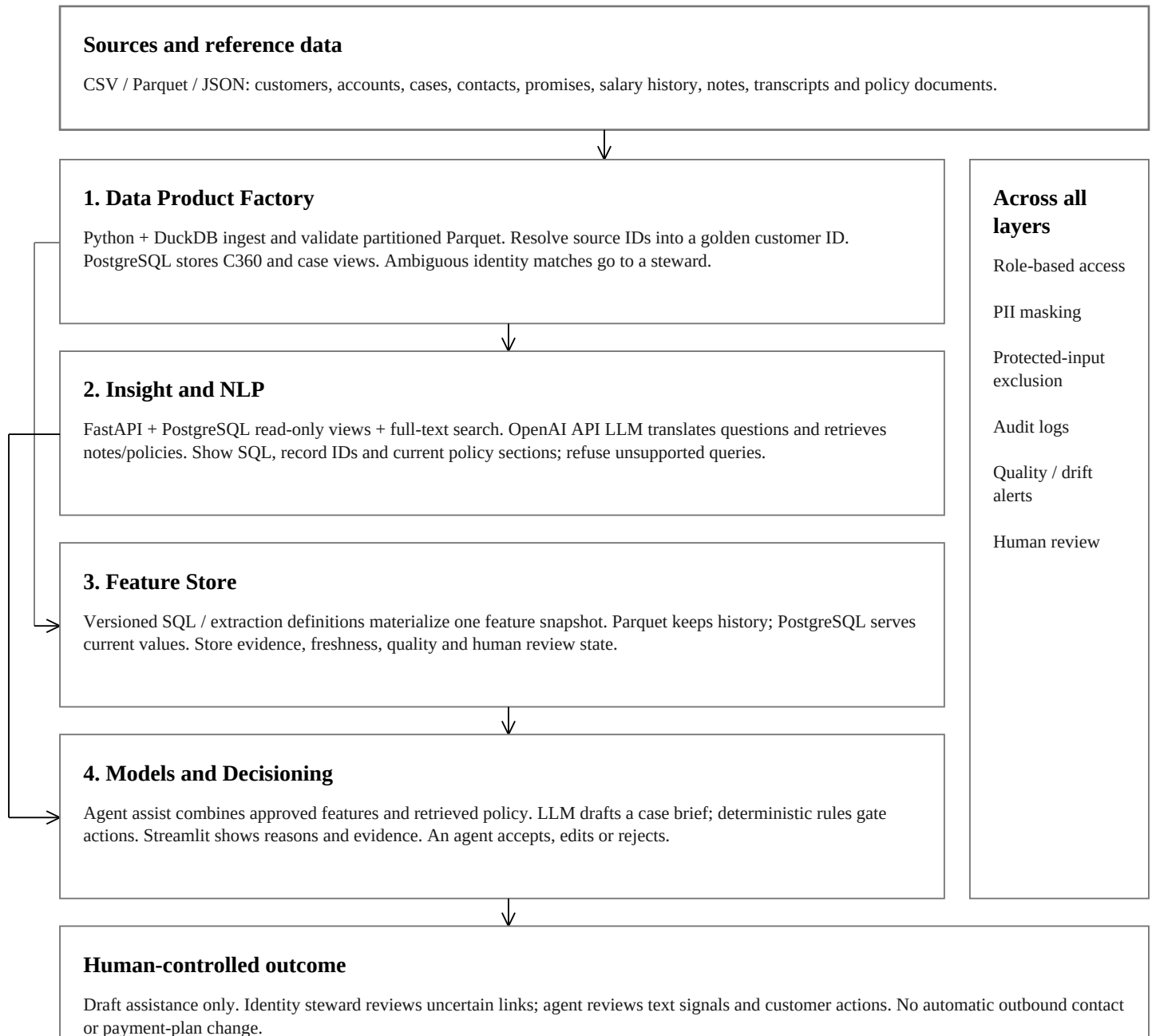
Collections Copilot

System design for agent assist

Team ⁵C₂

Jonathan Robin | Dhruva Anand | Atharva Desai | Ayaansh Solanki

Architecture diagram



Arrows show data flow: Layer 1 feeds Layers 2 and 3; Layer 2 supplies retrieved evidence to Layer 4; Layer 3 supplies features to Layer 4. AI is used in Layer 2, text extraction in Layer 3, and grounded drafting in Layer 4. Provider model, quotas and retention controls are TBC.

Problem and trusted data

Design document | Summary, business value, and Layers 1 and 2

Summary

Collections agents must assemble customer context from disconnected systems.

Our agent-assist copilot unifies records and answers questions with evidence.

Agents review case briefs; analysts inspect approved features and metrics.

Business value

Primary metric: median time to prepare an accurate case brief. Proposed target: a 30% reduction versus manual lookup, measured on the same synthetic cases in a timed crossover exercise. This is a target, not an achieved result. Also measure factual accuracy, valid source citations, unsupported claims, and recall of explicitly stated financial hardship on a human-labelled sample. Record review time and agent corrections. Recovery impact requires a later controlled pilot.

Layer 1

Ingest and curate. Keep immutable raw files and manifests. Validate schema, normalize phones, names, dates and codes, deduplicate by source key and snapshot, then publish accepted rows; quarantine failures. Handle the documented contact-history channel rename with a versioned mapping from channel_v2 or legacy channel. Conflicting values require review.

Resolve identity. Product src_customer_ref values are not CRM IDs. Map source-system/key pairs to golden_customer_id. Begin with normalized exact name + DOB + phone matches that identify one CRM candidate; do not merge on name or phone alone. CRM duplicates and conflicting candidates go to a steward. Retain match method, confidence category, source IDs and review history. DOB and other matching PII stay in the restricted identity area and never become decision inputs.

Publish C360. One customer row links to account, case and interaction child records. Keep separate account/case grains to avoid multiplying amounts through joins. Deduplicate account snapshots before aggregation; deposit balances are context, not debt or proof of affordability. Preserve source record, event time, availability time, refresh time and quality state for each derived field. Unresolved links cannot drive cross-product advice.

Layer 2

Classify questions as structured metrics or document/customer-context retrieval. Use metric_definitions for certified calculations. The LLM proposes structured parameters or SQL; SQLGlot permits one SELECT over allowlisted read-only views, with authorized customer scope, row limits and timeouts. Execution uses a restricted role. For notes and policy, retrieve authorized text with PostgreSQL full-text search before drafting an answer.

Every answer shows SQL or note/transcript IDs and policy section/version, the as-of date and any missing-data caveat.

Example: "What promise is currently open for this case?" returns amount and due date from promises_to_pay. Refuse requests for protected-attribute ranking, other agents' unauthorized customers, write actions, or evidence outside the dataset. Ask for clarification when the question is ambiguous; abstain when evidence is absent.

Feature store

Design document | Layer 3 (Feature Store)

Layer 3 (Feature Store)

The MVP exposes 12 features. Case features use `case_id`; customer features use `golden_customer_id`. A requested case receives its approved customer features through the identity map. All windows end at decision time `t`. Null means unknown; complete observed history with no events may produce zero.

Feature	Grain / refresh	Source	Definition and missing data
<code>case_max_dpd</code>	Case / daily	<code>collections_cases.current_dpd</code>	Days overdue for the worst account in this case; cross-check linked account DPD.
<code>case_overdue_cad</code>	Case / daily	<code>collections_cases.total_overdue</code>	Past-due CAD amount for the case; do not sum overlapping case totals.
<code>days_since_payment</code>	Case / daily	<code>card_accounts / loan_accounts.last_payment_date</code>	<code>t</code> minus latest payment date across linked case accounts; absent date = null.
<code>broken_ptp_90d</code>	Case / daily	<code>promises_to_pay.ptp_status, status_ts</code>	Count distinct promises first observed as broken in <code>[t-90d, t]</code> . History unavailable = null.
<code>kept_ptp_rate_90d</code>	Case / daily	<code>promises_to_pay.ptp_status, status_ts</code>	Kept / resolved promises in <code>[t-90d, t]</code> ; resolved = kept, partially_kept or broken. No resolved promises = null.
<code>open_ptp_days_to_due</code>	Case / on change	<code>current_ptp_id; promises_to_pay.ptp_due_date</code>	Due date minus local date at <code>t</code> . No open promise = null with <code>no_open_ptp</code> state.
<code>salary_decline_ratio</code>	Customer / monthly	<code>salary_credit_history credit_amount_total</code>	Aggregate customer salary across linked deposit accounts by month, then 1 - last complete month / prior 3-month mean. Missing/zero baseline = null.
<code>contact_attempts_7d</code>	Case / after contact	<code>contact_history.contact_id, contact_ts_utc</code>	Distinct outbound attempts in <code>[t-7d, t]</code> . Contact-policy gates also check customer-wide caps.
<code>channel_rpc_rate_90d</code>	Case / after contact	<code>contact_history.channel[_v2], rpc_flag</code>	Right-party contacts / observed outbound attempts in <code>[t-90d, t]</code> by channel; empty group = null.
<code>hardship_mention</code>	Case / after note	<code>agent_notes.note_text; call_transcripts turn.text</code>	Present / not_found / unknown, with supporting excerpt and review state. Absence is not proof of no hardship.
<code>callback_request</code>	Case / after note	<code>agent_notes.note_text; call_transcripts turn.text</code>	Extract requested date/window and channel. Ambiguous phrases stay unresolved for agent review.
<code>action_eligibility</code>	Case / at review	<code>customers.consent_*; case holds, caps, disputes</code>	Per-channel allowed / blocked / unknown with reasons. Current source flags govern, not LLM judgment.

One definition, two consumers. A versioned registry records SQL/prompt, type, grain, owner, source, refresh and permitted use. One materialization writes immutable Parquet history and the PostgreSQL current snapshot. Training uses point-in-time joins with `event_time AND available_at <= t`; live retrieval reads the same snapshot version. Reject future outcomes and unreviewed text signals as model inputs.

Late data and corrections. Preserve the original snapshot and availability time; append corrected versions. Do not backfill information into a past decision that did not have it. If historical promise status transitions are absent, mark the affected training features TBC/unavailable rather than using final outcomes. Verify parity on sampled snapshot IDs.

Agent assist and evidence

Design document | Layer 4, AI usage, and product experience

Layer 4 (Models and Decisioning)

Chosen use case: **agent assist**. The agent selects an authorized case. The system retrieves C360, current features, approved note excerpts and applicable policy. The LLM drafts a concise case brief and a proposed assistance step. Deterministic rules check current consent, cease-contact requests, contact caps, disputes and operational holds before displaying any contact option. Unknown or stale permissions block the option pending refresh/review.

Output includes the proposed step, reason codes, supporting feature values and timestamps, note/transcript evidence, policy section/version, and unresolved questions. A recorded hardship flag or a pending hardship mention prompts specialist review, never harsher treatment. Financial offers require the policy-defined approver. The agent accepts, edits or rejects the draft; the system records the reviewer and changes. The prototype never sends a message or modifies a repayment arrangement.

AI usage

Component	Why AI is useful	Control and evaluation
Question interpretation	Translate ordinary language into certified metrics or scoped retrieval.	Validate query structure; test benchmark_questions, including refusal cases.
Text feature extraction	Find explicit financial-hardship mentions and callback requests in messy notes/transcripts.	Schema-constrained JSON; evidence must match a source span. No inferred protected traits. Agent confirms or corrects.
Case brief drafting	Summarize relevant history and explain assistance in plain language.	Use retrieved evidence only; cite each factual claim. Compare against human-reviewed case briefs.

Use an OpenAI API LLM with structured outputs; exact model and data-handling settings are TBC. Redact direct identifiers and protected-attribute mentions before API calls. Treat retrieved text as data, not instructions. Unsupported extraction, conflicting evidence or an API failure produces an explicit unknown state and a source-record view for manual review.

Product experience

Feature catalogue. Search approved features; inspect definitions, grain, owner, version, permitted use and refresh schedule. Reusable sets select approved features for training and live retrieval.

Customer feature viewer. See current values, historical as-of snapshots, source coverage, stale values and conflicts.

Evidence and review panel. Open redacted source spans; confirm/correct extracted signals. Corrections retain the prior value, reviewer and timestamp.

Example interaction

Illustrative synthetic statement: "My salary is delayed; please call Friday." The extractor creates a pending hardship mention and callback request, each linked to its source words. It does not invent a calendar date for an ambiguous Friday. The agent clarifies timing and reviews assistance policy. A follow-up draft is available only if current contact rules permit it. A salary decline is context for a question, not a conclusion about affordability.

Delivery and controls

Design document | Scalability, governance, and build plan

Scalability

The dictionary describes about 1M customers, 1.84M accounts and much larger history tables. Batch Python/DuckDB jobs scan only required columns and date partitions in Parquet, publish C360 and feature snapshots, and process only changed notes. PostgreSQL serves indexed customer/case lookups and approved aggregates; historical scans run outside the live request path. Text extraction is asynchronous, bounded and cached by source hash plus prompt version.

Daily jobs refresh financial aggregates. Contact ingestion refreshes affected cases; consent and holds are rechecked at review time. Monthly jobs calculate income-change features after a complete month. FastAPI separates retrieval from batch work; Streamlit is the demonstration UI. Track query latency, extraction queue age, stale-feature rate and database load. Scale API replicas and batch partitions as measured demand grows. No real-time latency or throughput claim is made before the dataset benchmark.

Governance

Access and privacy. Enforce agent/customer assignment and analyst aggregate permissions server-side; use least-privilege roles, masked PII, encrypted transport/storage and protected audit logs. Separate the restricted identity map from approved decision features. Authorized policy retrieval respects document applicability and effective version.

Responsible decisions. Explicitly exclude protected attributes listed in the dictionary and their direct derivatives from SQL decision views, feature sets and LLM decision context. Review potential proxies. Financial hardship leads to assistance and human review; never infer protected traits or affordability from voice, name, location, sentiment or deposit balance. Optional voice is deferred because transcripts already support the use case.

Review and monitoring. Humans resolve uncertain identity matches, confirm text signals, and approve customer-affecting proposals. Log snapshot, definition/prompt version, evidence, reason codes and overrides. Monitor schema changes, match conflicts, feature drift, missingness, unsupported claims and refusal failures. Failed quality checks quarantine data; stale or unknown constraints produce abstention. Retention period, production security approvals and precise policy thresholds are TBC.

Build plan

Hours	Deliverable	Acceptance check
0-6	Load priority tables; schema adapter; identity map; C360 and data contract.	Deduplication and linkage checks; ambiguous matches visibly quarantined.
6-14	Materialize 12 features; registry; current/history storage; catalogue and customer viewer.	Window, null and point-in-time checks; sampled offline/live snapshot parity.
14-22	Scoped question interface and grounded notes/policy retrieval.	Run available benchmark questions; show SQL/sources and refusal results.
22-30	Text extraction, case brief and evidence/review panel.	Manual-labelled extraction sample; holds, consent and stale-data scenarios block unsafe drafts.
30-36	Integration, measured evaluation, quality report and final demo materials.	End-to-end case with evidence and human override; report failures alongside metrics.

Build priority covers the four required layers and one use case. Autonomous outreach, voice processing and predictive treatment optimization are future extensions. API credentials, dataset completeness and policy details will be confirmed when the build phase starts.

Draft data contract

One-page contract | Collections 360 and approved feature snapshot | v0.1

Product owner: Collections Analytics. **Technical steward:** team ⁵C₂ during the prototype; production owner TBC. **Consumers:** authorized collections agents, analysts and approved model builders.

Grain: one golden customer per C360 snapshot; one case or customer per feature snapshot. IDs below are proposed output fields, not assumed source columns.

Output field	Type / constraint	Meaning
golden_customer_id	string, required	Stable internal ID; connects approved views to the restricted identity map.
case_id	string, nullable by entity	Required for case-level features; null only for customer-level snapshots.
entity_grain	enum: customer / case	Determines entity key and prevents mixing customer and case aggregates.
snapshot_id	string, required	Immutable snapshot identifier shared by history and current serving.
as_of_ts / available_at	UTC timestamps, required	Decision cutoff and time this snapshot became usable.
feature_set_version	string, required	Approved definition bundle, including SQL and extraction-prompt versions.
values / value_states	typed feature map / enum map	The 12 features on page 3; known, unknown, no_open_ptp or not_applicable states.
lineage_refs	array, required	Source table/key, event and ingestion times; transformations and policy versions.
review_state	enum per text signal	pending / confirmed / corrected / rejected; reviewer and review time when resolved.
quality_state / refreshed_at	enum / UTC timestamp	accepted / quarantined / stale; last successful refresh.

Quality rules and failure handling

- C360 key (golden_customer_id, as_of_ts) and feature key (entity_grain, entity_id, as_of_ts, feature_set_version) must be unique; entity_id means the appropriate customer/case key.
- Every published case/account link must resolve to an accepted identity mapping; ambiguous links are quarantined.
- DPD/counts are nonnegative integers; kept-promise and RPC rates lie in [0,1]. Salary decline may be negative. Money is decimal CAD. Quarantine invalid rows.
- Historical feature inputs must satisfy event_time and available_at <= as_of_ts. No future payment outcomes or retrospective text corrections enter old training rows.
- Every AI-extracted signal must cite a verifiable source span and extraction version. Pending/rejected signals are excluded from approved training sets.
- Required consent/hold checks must be current at action review. Unknown, stale or conflicting permission values block the proposal.

Allowed uses, refresh and change control

Allowed uses are authorized case preparation, certified aggregate analysis and approved training/evaluation on synthetic data. Prohibited uses include protected-attribute decisions, marketing, unauthorized data exports, autonomous outreach and automatic repayment-plan changes. Raw PII and matching-only fields are excluded from exported feature sets.

Proposed refresh: financial facts daily with a 24-hour freshness limit; contact/text features within 5 minutes of ingestion; income monthly after completeness checks; consent/holds read at review time. These are design targets; actual upstream SLAs are TBC. Preserve last-good snapshots with a visible stale state; fail closed on action gates. Version schema and definitions, publish breaking changes with a migration plan, and log corrections. Retention and deletion schedules are TBC.

Design basis: CIBC Collections Hackathon Problem Statement-3.pdf, pp. 1-3; CIBC Hackathon - Data Dictionary.xlsx, About, Tables, Columns and Relationships; collection hackathon.pdf, architecture and feature-store examples. Proposed measures, schemas and targets have not yet been validated on the unreleased dataset.